Anonymity and Privacy on the Internet

Lecture 32

Computer Networks and Distributed Programming

Where we are: what encryption leaves visible

- We ended with a sniffer on port 443: one name readable, and everything else Application Data.
- Still readable by anyone on the path: the addresses, the server name, the sizes, the timing.
- So a sniffer learns who talks to whom, and when, even when it reads nothing of what they say.
- TLS hides what you say. Today: whether anything can hide whom you say it to.
- The plan: whom you hide from; what each layer gives away; the tools; what no tool can fix.

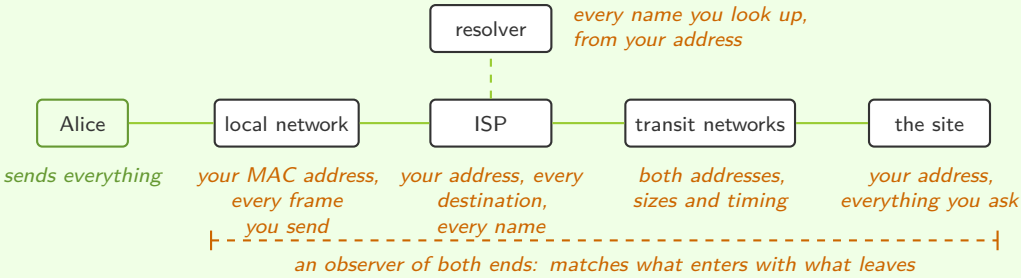
Three things Alice wants to hide

Alice wants to read a political activist's site. She does not want:

	HTTP	HTTPS
the site to learn her IP address	exposed	exposed
her ISP to learn which site she visits	exposed	exposed
her ISP to read what she exchanges	exposed	hidden

- Her source address is in every datagram she sends; the site's address is in every datagram too.
- TLS fixes one of the three. By the end of the hour this table will have two more columns.

Anonymous with respect to whom



- There is no being anonymous in general: only anonymous with respect to someone.
- Each observer sits at a different point and sees a different part. The dangerous one sees both ends.

Reading and blocking are different threats

- An observer wants to read or to link: who you are, whom you talk to, what you say.
- A censor wants neither. It wants to stop the conversation, and only needs to recognise it.
- By address: drop every packet towards a list of IP addresses: the packet filter we wrote rules for.
- By name: answer DNS queries with lies, or reset connections whose SNI is on a list.
- By shape: deep packet inspection recognises a protocol even when it cannot read a byte of it.
- Hiding who you are does not stop a censor. Hiding what the traffic looks like might.

Four different things

Four properties

Confidentiality: nobody but the two ends can read the content.

Anonymity: you cannot be identified within a set of subjects, the anonymity set.

Pseudonymity: you act under an identifier that is not your name — but is the same every time.

Unlinkability: nobody can tell whether two messages, or two visits, come from the same subject.

- They are independent. A pseudonym is linkable by design: that is what it is for.
- Confidentiality says nothing about the other three, and it is the only one TLS provides.

The pseudonym a court unmasked

A remailer receives your mail, removes your address, and reposts it under a pseudonym. Who can find out it was you?

Whoever holds its table from pseudonyms to addresses — and whoever can force the operator to hand it over. In the 1990s a court did exactly that.

- Pseudonymity is anonymity only as long as the mapping stays secret: a type 1 remailer.
- Cypherpunk remailers kept no table and no logs: a seized server had nothing to give.
- Users chained three of them, each message encrypted for the next. We will meet that shape again.

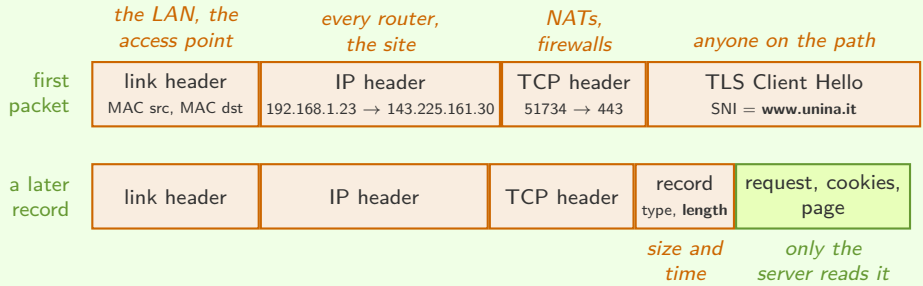
A padlock and a login

You log in to a site over HTTPS. Which of the four properties do you have?

Only confidentiality, and only against the path. The site has your name, and your ISP knows which site it is.

- TLS authenticates the server; your login authenticates you. Together they are the opposite of anonymity.
- Taking encryption for anonymity is the most common mistake about privacy on the Internet.
- And every layer under TLS gives something away. What, and to whom, we see next, one layer at a time.

One HTTPS request, read from outside



- TLS encrypts its own payload. Every header below it travels exactly as it was written.
- Even the encrypted part tells its length, and the moment it was sent.

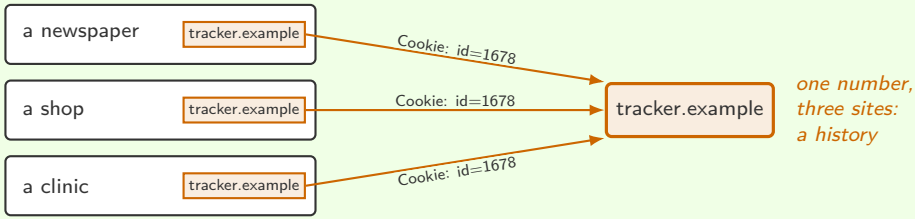
A header is written to be read

- Below its own layer a payload is opaque. A header is the opposite: it exists to be read on the way.
- The switch must read the MAC address, every router the IP address, the NAT the ports, the resolver the name.
- So a layer can only encrypt what lies above it; its own header stays in clear for its own devices.
- TLS sits above TCP: from the ports down, everything is in clear — and has to be.
- Each layer gives away exactly what it needs to deliver. Going down the stack, the leak changes owner.

What you tell the application

- At the top, you often say who you are: an account, a login, a delivery address, a phone number.
- No layer underneath can undo that. The best network anonymity ends at the login page.
- Without a login there is the cookie: a number the server gave you, and you send back with every request.
- That number is a pseudonym: it links all your visits without needing your name.
- As we saw with cookies: give your name once, and the whole history acquires one.

Third-party cookies: one identifier across many sites



- A page embeds an image or a script from another domain, and that domain sets a cookie of its own.
- The same cookie comes back from every site that embeds it, together with the site you were on.
- Browsers now isolate such cookies per site, or block them — to different degrees.

No cookies, and still recognised

You delete every cookie and come back. Can the site still tell you apart from everyone else?

Often, yes. User agent, screen size, fonts, time zone, language, how the graphics card draws a test image: together they are nearly unique, a fingerprint.

- It is sent without asking, needs nothing stored on your machine, and deleting data does not reset it.
- Changing values nobody else has is a mark of its own. The defence is uniformity: every user reports the same.
- Tor Browser does exactly that. We will see why it has to.

Every name, before every connection

- Before a connection there is a query, in clear on UDP port 53, carrying the name you are about to visit.
- It crosses the local network and the ISP, and reaches the resolver: by default, the one DHCP gave you — the ISP's.
- The resolver sees every name you look up, from your address, with the time. It may keep them.
- As we saw with DNS security, DoT and DoH hide the query from the path.
- Not from the resolver: it still has to answer, and so it still reads.

Encrypted DNS: who reads the names now

You switch to DoH towards a large public resolver. Who knows which names you look up?

The public resolver, all of them. Your ISP no longer sees the queries — and a moment later sees the SNI and the address of the connection that follows.

- Encrypting a channel does not remove the observer at its far end: it moves it.
- You traded a resolver you did not choose for one you did, and now you must trust that one.
- Keep this in mind: it is the pattern of every tool in this lecture.

What TLS leaves in the clear

- The name: the SNI in the Client Hello. ECH encrypts it, with a key the site publishes in the DNS.
- The address: of the destination, in every datagram: TLS sits above IP and cannot touch it.
- The port: which usually says the application: 443 the Web, 853 DNS over TLS, 993 mail.
- Sizes and timing: a page is a sequence of objects of known lengths: its shape survives encryption.
- Matching encrypted shapes to known pages is website fingerprinting. It works — less well — even through Tor.

A hidden name needs a crowd of names

With ECH the name is encrypted and only the address is left. When does the address give the site away anyway?

When the site has an address of its own. Behind a large CDN, thousands of names share the same addresses: the address only says which CDN.

- ECH hides a name only among the other names behind the same front.
- It also changes a censor's arithmetic: blocking one site means blocking all of them.
- Both ideas — hiding in a crowd, being expensive to block — come back with Tor.

Why not forge the source address

The source address identifies you to everyone on the path. Why not simply write a false one?

Because it is also the return address. The reply goes to whoever owns the forged one, and your connection never completes.

- Forging works only when no answer is wanted: the reflection attacks we saw with spoofing.
- TCP needs the SYN-ACK back, and every request expects content in return.
- To hide the return address you need someone who receives the reply for you, and passes it on: a proxy.

An address points to a contract

- Your ISP assigned the address, and knows which subscriber had it at what time. A court can ask.
- Address blocks are registered by region: a public address locates you, roughly, for anyone.
- NAT: the whole house leaves with one address: you are hidden among your family, not the world.
- Shared address space: the provider's translator hides you among its customers — and the provider holds the table.
- IPv6: one global address per device, and no crowd; hosts pick temporary addresses and keep changing them.

The MAC address never leaves the LAN

- Every router discards the frame and builds a new one: the site never sees your MAC address.
- Everyone on your LAN does, and so does whoever runs the access point: the café, the hotel, the campus.
- A fixed MAC recognises the same device in every network it joins: hence the random local addresses we have met.
- Many systems keep one random address per network, so that network still recognises you when you return.
- And the DHCP request often carries the machine's host name, as its owner set it.

A transmitter gives away where it is

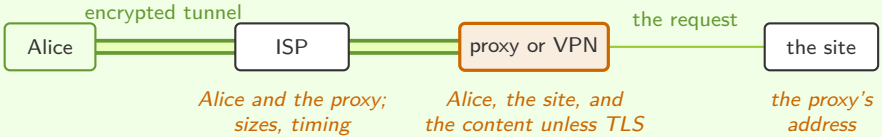
- Down here the leak is physics, not a header: to be received, you have to radiate.
- Power falls with distance, so receivers comparing strength and direction locate a transmitter.
- The cellular network must know your cell to deliver a call: location is part of the service.
- A phone scanning for known WiFi networks announces itself to every access point in range.
- No encryption hides that a transmission took place, from where, and when.

Who sees what, down the stack

	What leaks	**Who sees it**	**What reduces it**
Application	login, cookies, fingerprint	the site, embedded third parties	no login, isolated cookies, uniformity
DNS	every name	the path, the resolver	DoT, DoH: the resolver remains
TLS, TCP	name, port, sizes, timing	anyone on the path	ECH, in a crowd
Network	both addresses	routers, the site, the ISP's records	a proxy: someone else's address
Link	MAC, host name	the LAN, the access point	a random MAC per network
Physical	that you transmit, and where	anyone in range	only silence

Every remedy moves an observer, or shrinks what it sees. None removes it.

One hop: the anonymizing proxy



- Alice opens TLS to a proxy, which makes the request for her: the institutional proxy, used the other way round.
- A VPN does it for all traffic: each datagram encapsulated, encrypted, inside another — a tunnel.
- Either way the proxy is a man in the middle you chose. It sees what the ISP saw, and more.

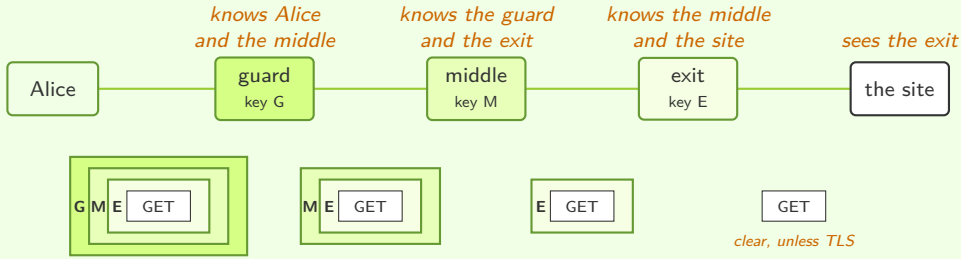
The root server that answered too well

Behind one VPN, a root server returned an address it could not have had. Who answered?

The VPN provider, in the root server's name. It sits on the path of everything you send, and can answer as anyone.

- You replaced the ISP with an observer that sees the same, and also has your account and your payment.
- "No logs" is a promise about its behaviour, not a mechanism you can check.
- Right against the local network and the ISP; useless against the provider itself.
- A company VPN is another thing: it exists to let you in, not to hide you.

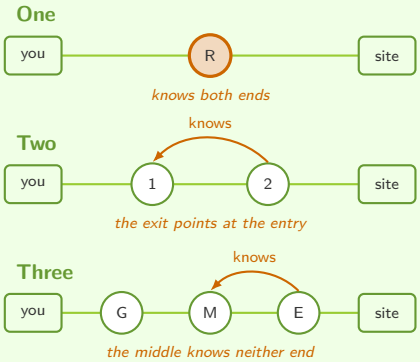
Tor: a circuit of three relays



each relay removes the layer of its own key, and forwards the rest

- The client picks three volunteer relays, and agrees a key with each of them by Diffie-Hellman.
- It wraps the request in three layers. Each relay removes its own and forwards the rest: encapsulation, one key per layer.

Why three relays



- One: the relay knows both ends: it is the proxy, trusted with everything.
- Two: neither knows both ends — but the exit knows the entry: exactly the node to watch.
- Three: the exit knows only a middle, and the middle knows neither end.
- Three does not stop a guard and an exit working together: they match the timing.

Tor, seen by the same sniffer

```
$ sudo tshark -i enp3s0 -Y tls          then start tor, and:  torsocks curl -sI https://www.unina.it
  3 0.0402 192.168.1.23 -> 198.51.100.23 TLSv1.3  571 Client Hello (SNI=www.k3fq7zn2tb.com)
  5 0.1296 198.51.100.23 -> 192.168.1.23 TLSv1.3 1514 Server Hello, Change Cipher Spec, Application Data
  9 0.2615 192.168.1.23 -> 198.51.100.23 TLSv1.3  602 Application Data
 11 0.3907 198.51.100.23 -> 192.168.1.23 TLSv1.3  602 Application Data
 14 0.5213 192.168.1.23 -> 198.51.100.23 TLSv1.3  602 Application Data
 17 0.6698 198.51.100.23 -> 192.168.1.23 TLSv1.3 1204 Application Data
```

- Every packet goes to one address, the guard. `www.unina.it` is nowhere — the SNI is a name Tor made up.
- Records of the same few lengths: Tor carries data in fixed-size units, so sizes say much less.
- What the ISP still learns: that you use Tor, when, and how much. The list of relays is public.

The exit relay reads what it carries

There is no TLS between Tor and the site. What does the exit relay see?

What a network on the site's side of the path would see: the request, the reply, any password in them — but not who you are, unless the content says it.

- Exits are run by volunteers, and some have been caught rewriting plain HTTP on its way.
- With TLS on top, the exit sees the name and the address, and no more: what your ISP sees without Tor.
- Tor hides who; TLS hides what. You need both.

Traffic through Tor, names through the ISP

An application sends its connections through Tor but resolves names by itself. What does the ISP see?

Every name, in clear, just before each connection. The traffic is hidden; the list of destinations is not.

- Tor resolves names at the exit: the client hands it the name, never an address it looked up.
- Tor carries only TCP. Whatever an application sends over UDP goes around it, from your address.
- A leak is anything that leaves by another way. Tor protects what it carries, and nothing else.

A server nobody can locate

- So far the client hides and the server is public. An onion service hides the server as well.
- The server keeps circuits to a few relays, its introduction points, and publishes them signed.
- Client and server each build a circuit to a rendezvous relay, which joins them: six relays, and no exit.
- No exit means that no relay ever holds the traffic in clear.
- The `.onion` name encodes the server's public key: no DNS to ask, no CA to trust.
- Deep and dark: the deep Web is what search engines do not index; the dark Web, what only such networks reach.

Blocking the relays: bridges

- To choose three relays a client must know them all: the list of relays is public, by design.
- A censor copies it, and drops every packet towards those addresses: the packet filter we wrote rules for.
- A bridge is an entry relay that is not on the list, handed out a few at a time.
- The censor can ask for bridges too, and block those it gets: distribution is a race.
- A bridge hides where you connect. It does not hide what the connection looks like.

Blocking the shape: pluggable transports

DPI recognises Tor's handshake, so a transport makes it look like random bytes. Why can that still give it away?

Because almost everything else is TLS now. Bytes that look like nothing stand out: the anomaly-based detection we saw with intrusion detection.

- Pluggable transports: reshape the traffic between client and bridge, not what it carries.
- Better: look like something common, an HTTPS connection or a video call — or hide behind a CDN too costly to block.
- Steganography applied to traffic, and an arms race: every disguise is studied until it is recognised.

I2P: garlic routing

	Tor	**I2P**
Who relays	volunteer relays; most users only connect	every participant routes for others
Paths	one circuit, both directions	separate one-way tunnels, out and in
A layer carries	one message	several messages bundled: a garlic
Built for	reaching the ordinary Internet	services inside the network

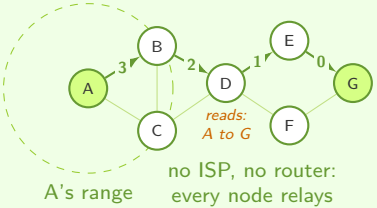
- The same idea — layered encryption over relays the client chooses — with different bets on who relays.
- Separate tunnels mean that the way out and the way back cross different relays.

Protecting the machine, not the path

- All of the above protects the path. A different problem: what the machine itself keeps, and lets out.
- Tails: a system started from a USB stick, that sends everything through Tor and forgets everything at shutdown.
- Qubes: each activity in its own virtual machine: one broken compartment does not see the others.
- GrapheneOS: a hardened Android: stricter permissions, apps kept apart, less to attack.
- Whonix: two machines: the workstation reaches the network only through a gateway that runs Tor.
- That is the application gateway again: malware inside cannot leak your address, because it has no route.

Going around the infrastructure: mesh radio

- Every tool so far crosses the infrastructure. Where a state blocks it, go around it.
- LoRa: a physical layer: spread chirps in sub-GHz ISM bands, kilometres at low power.
- LoRaWAN: a star on top of it: devices, gateways, a server — infrastructure again.
- Meshtastic: a mesh: each node rebroadcasts what it hears, with a hop limit, like the TTL.
- Peer-to-peer did it one layer up: nothing to block — and every peer saw the others' addresses.



What a mesh costs, and what it exposes

- Bandwidth: a narrow channel and a weak signal: by Shannon, hundreds to thousands of bit/s, a few hundred bytes a message.
- Air time: unlicensed bands cap how long you transmit: in Europe about 1% of the time, in most of the band.
- Flooding: every message reaches every node: the whole mesh shares one capacity, which does not grow with it.
- Headers: every relay must read sender, destination and hop count: who talks to whom is public in range.
- Content is encrypted with a key the channel shares — the default one is public — and radio amateurs may not encrypt.
- And it radiates: useful where a state blocks; a beacon where it hunts transmitters.

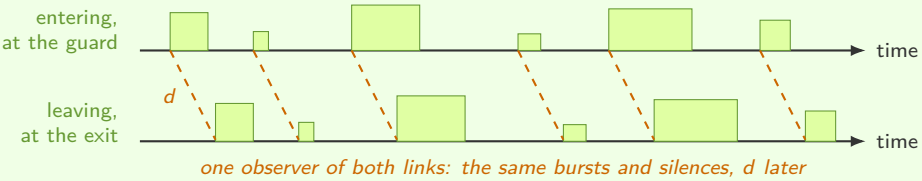
If three relays are good, why not ten?

The third relay helped. Why does Tor not use ten?

Because the strongest attack ignores the middle. Whoever watches the traffic entering at the guard and leaving at the exit matches the two, however many relays lie between.

- Each extra relay adds latency, and one more chance of a slow or hostile one — and takes nothing from that attack.
- Three buys what relays can buy. The rest is a limit of the design, not of the count.

The same pattern at both ends



- Encryption changes the bytes, not when they move or how many: bursts and silences cross the circuit intact.
- Breaking the match means holding, padding and reordering traffic, as the remailers did: mix networks.
- That costs seconds or hours. Tor chose low latency, so the Web works — and so it can be correlated.

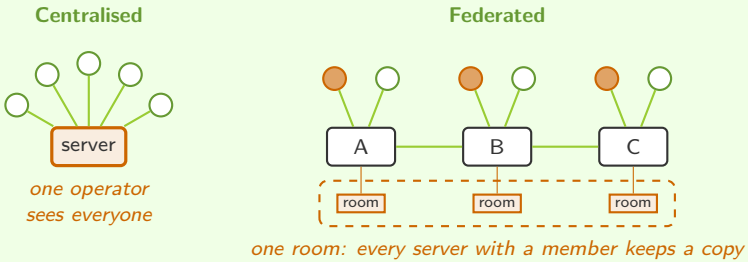
Who can see both ends

- A national ISP, when the user and the site are in the same country.
- An autonomous system lying on both paths — and which ones do, BGP decides, not you.
- Whoever runs many relays, and is sometimes chosen as both the guard and the exit.
- Hence the guard: a client keeps the same entry for months. Choosing afresh each time makes a hostile one certain, sooner or later.
- Tor never claimed otherwise: from its first design, it does not protect against an observer of both ends.

Encrypted content, visible conversations

- Signal and Matrix encrypt messages end to end: the server stores what it cannot read.
- But it delivers them, so it sees who connects, when, how often, how much — and usually to whom.
- Signal is built to keep as little as possible, and to hide even the sender from its own server.
- The network adds its share: your address reaches the server with every message.
- Talking to Alice puts Bob on the list: repressive states understand transitive closure.
- For much surveillance who talks to whom is enough, and it is far cheaper to store than content.

Where the metadata lives: federated or centralised



- Centralised: one operator sees the metadata of everyone.
- Federated: like mail: each user on a server, the servers talk to one another. Matrix is federated.
- Your own server keeps your metadata — until you join a room: every server with a member keeps a copy of it.

Tor and a credit card

You reach a shop through Tor and pay by credit card. How anonymous was the purchase?

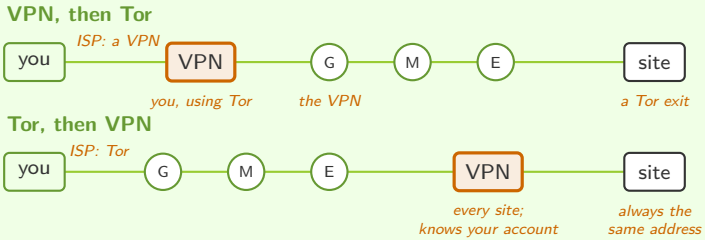
Not at all: the card carries your name. Pay in bitcoin and the shop gets an address on a ledger that is public forever.

- Every payment to and from that address is there for anyone to read. One link to a person names the whole history.
- The link is usually made where the coins were bought: an exchange that asked for your ID.
- Cryptocurrencies are pseudonymous, like cookies: give your name once, and every past payment acquires it.
- Monero hides sender, receiver and amount by default; Zcash does when the user chooses to.

Anonymity needs a crowd

- Anonymity is always among others, the anonymity set. Alone, the only candidate is you.
- Tor needs many users, and different ones: if only dissidents used it, using it would be a confession.
- A VPN server of your own has one user: everything leaving its address is you.
- A commercial VPN has a crowd — but its provider sees through it.
- Tor Browser makes its users look identical: the answer to fingerprinting, and the reason not to customise it.
- Every user who joins protects the others. Anonymity is a collective good.

VPN and Tor: in which order



- VPN, then Tor: hides Tor from your ISP; the provider learns you use it. A bridge does that without a provider.
- Tor, then VPN: the site always sees one address; the VPN is an exit that knows who pays for it.
- Neither is safer in general. The question is the first one of the hour: from whom?

It depends on whom you hide from

Alice does not want	**HTTP**	**HTTPS**	**VPN**	**Tor**
the site to learn her IP address	exposed	exposed	hidden	hidden
her ISP to learn which site she visits	exposed	exposed	hidden	hidden
her ISP to read what she exchanges	exposed	hidden	hidden	hidden
any single party to know both ends	exposed	exposed	exposed	hidden
an observer of both ends to match them	exposed	exposed	exposed	exposed

- Read it by rows: each row is an adversary, and each needs its own answer.
- A censor is not in it. Against a censor: bridges, disguises, and where those fail, a mesh.

Summary

- Nobody is anonymous in general, only with respect to someone; and the four properties are four, not one.
- Every header is written to be read: each layer, down to the radio, gives away what it needs to deliver.
- Every tool moves the observer. A VPN replaces it; Tor splits it over three relays that each know two neighbours.
- Censors block rather than read: bridges hide where, transports hide how, a mesh avoids the network.
- Nothing stops an observer of both ends, a server keeping metadata, or a payment in your name.
- We began by following one HTTP request across the network. We end knowing who can tell it was yours.

Exercise: a Tor with two relays

Alice wants a page of `activist.com` through two non-colluding proxies, Proxy1 and Proxy2. She has their certificates, with public keys K_1^+ and K_2^+. Draw each answer as a timing diagram.

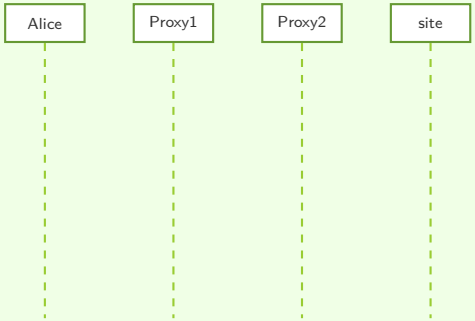
(a) A protocol, as simple as possible, by which Alice shares a session key S_1 with Proxy1.

(b) One by which she shares S_2 with Proxy2 without revealing her IP address to Proxy2.

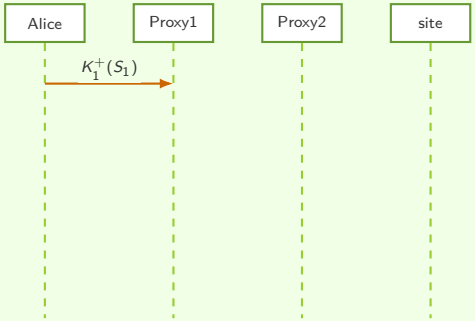
(c) With S_1 and S_2 in place and no public-key operations: a request for the page that hides Alice's address from Proxy2 and the site from Proxy1. End with the HTTP request arriving at `activist.com`.

(d) By the argument of this hour, what would a third proxy add? And what is lost if Proxy1's private key is stolen next year?

Exercise: a solution

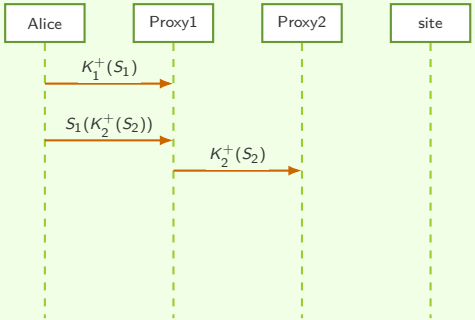


Exercise: a solution



(a) Alice picks S_1 and sends $K_1^+(S_1)$. Only Proxy1 can open it.

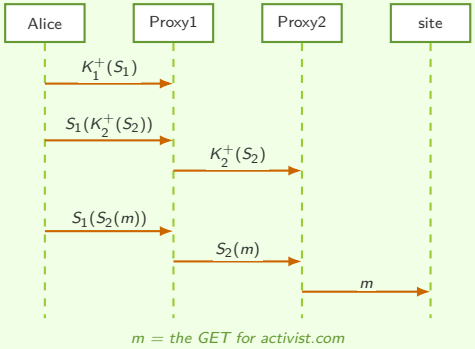
Exercise: a solution



(a) Alice picks S_1 and sends $K_1^+(S_1)$. Only Proxy1 can open it.

(b) She sends $S_1(\text{Proxy2}, K_2^+(S_2))$. Proxy1 peels its layer and forwards $K_2^+(S_2)$: Proxy2 sees Proxy1's address.

Exercise: a solution

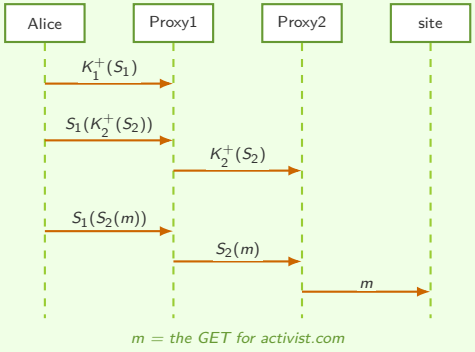


(a) Alice picks S_1 and sends $K_1^+(S_1)$. Only Proxy1 can open it.

(b) She sends $S_1(\text{Proxy2}, K_2^+(S_2))$. Proxy1 peels its layer and forwards $K_2^+(S_2)$: Proxy2 sees Proxy1's address.

(c) $S_1(\text{Proxy2}, S_2(\text{activist.com}, \text{GET}))$. Proxy1 learns only Proxy2, Proxy2 only the site; the reply comes back wrapped the other way.

Exercise: a solution



(a) Alice picks S_1 and sends $K_1^+(S_1)$. Only Proxy1 can open it.

(b) She sends $S_1(\text{Proxy2}, K_2^+(S_2))$. Proxy1 peels its layer and forwards $K_2^+(S_2)$: Proxy2 sees Proxy1's address.

(c) $S_1(\text{Proxy2}, S_2(\text{activist.com}, \text{GET}))$. Proxy1 learns only Proxy2, Proxy2 only the site; the reply comes back wrapped the other way.

(d) Proxy2 knows Proxy1: the node to watch. A third relay leaves the exit knowing only a middle. And with S_1 sent under K_1^+, a stolen key opens every recorded session: Tor uses Diffie-Hellman.